



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ITWM106

Mobile Application Development 3 (cross-platform)

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

Handling Input, Events, and Forms

ccms@mseuf.edu.ph @mseufCCMS www.mseuf.edu.ph

What Is User Input?

- User input is information provided by the person using the application.
- Examples include:
 - Name
 - Email
 - Password
 - Address
 - Phone number
 - Search keywords
 - Quantity
 - Comments
 - Feedback



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph



@mseufCCMS

www.mseuf.edu.ph

TextInput

- TextInput is a Core Component that allows the user to enter text.
- It has an **onChangeText** prop that takes a function to be called every time the text changed, and an **onSubmitEditing** prop that takes a function to be called when the text is submitted.
- Syntax:

```
<TextInput placeholder='Username' />
```



MANUEL S. ENVERGA UNIVERSITY FOUNDATION
COLLEGE OF COMPUTING & MULTIMEDIA STUDIES
CHED CENTER OF DEVELOPMENT IN INFORMATION TECHNOLOGY

ccms@mseuf.edu.ph



@mseufCCMS

www.mseuf.edu.ph

TextInput

- **placeholder** - string that will be rendered before text input has been entered.
- **onChangeText** - callback that is called when the text input's text changes. Changed text is passed as a single string argument to the callback handler.
- **secureTextEntry** - If true, the text input obscures the text entered so that sensitive text like passwords stay secure. The default value is false
- **value** – current value of the TextInput

TextInput

- TextInput has a built in virtual keyboard that can be configured. To set the type of keyboard for the TextInput, use the keyboardType property
- **keyboardType** - Determines which keyboard to open, e.g.numeric
 - all platforms (default, number-pad, decimal-pad, numeric, email-address, phone-pad)
 - iOS only (ascii-capable, numbers-and-punctuation, url, name-phone-pad, twitter, web-search)
 - android only (visible-password)

TextInput

Sample output:

```
<TextInput
  style={styles.textInput}
  placeholder='Enter Num 1'
  onChangeText={({text) => setNum1(parseFloat(text))}
  value = { num1.toString() }
/>
```

TextInput

Sample output:

```
<TextInput
  style={styles.textInput}
  placeholder='Enter Num 1'
  onChangeText={({text) => setNum1(parseFloat(text))}
  value = { num1.toString() }
/>
<TextInput placeholder="Password" secureTextEntry />
```

TextInput

- For the documentation on TextInput go to <https://reactnative.dev/docs/textinput>

Handling Events

- An event is an action triggered by the user, such as tapping, typing, or scrolling.
- In React Native, events are handled using props that take a function, which is called automatically whenever that action happens.
 - `onPress` – called when a component is tapped
 - `onChangeText` – called when a TextInput's text changes
 - `onSubmitEditing` – called when the keyboard's submit/return key is pressed
 - `onFocus` / `onBlur` – called when an input gains or loses focus

Handling Events

- Button is a Core Component that renders a simple, platform-styled button that responds to taps.
- It requires two essential props:
 - title – the text displayed inside the button
 - onPress – the function called when the button is tapped
- Syntax:

```
<Button title="Submit"
  onPress={() => Alert.alert('Tapped!')} />
```

Handling Events

- TouchableOpacity wraps any component (View, Text, Image) and makes it respond to touch, fading briefly when pressed.
- Use it for a custom-styled button that Button cannot provide, such as custom colors, icons, or layout.
- Syntax:

```
<TouchableOpacity onPress={() => setCount(count + 1)}>
  <Text>Tap Me</Text>
</TouchableOpacity>
```

Building a Form

- A form is a group of input fields that collects related information from the user, typically submitted together.
- Common examples include:
 - Login form – username, password
 - Registration form – name, email, password, confirm password
 - Feedback form – name, comments, rating
- In React Native, a form is simply a set of TextInput components combined with a Button, all managed using state.

Building a Form

- To build a form, create a state variable per field with useState, then bind it to the matching TextInput's value and onChangeText.

```

const [username, setUsername] = useState("");
const [password, setPassword] = useState("");
<TextInput value={username}
  onChangeText={setUsername} />
<TextInput secureTextEntry value={password}
  onChangeText={setPassword} />
  
```

Building a Form – Full Example (p.1)

- A login form ties everything together: state for each field, and a handleSubmit function that runs when the button is pressed.

```
import React, { useState } from 'react';
import { View, TextInput, Button, Alert } from 'react-native';

const LoginForm = () => {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
```

Building a Form – Full Example (p.2)

- handleSubmit validates the fields, then uses Alert to give the user feedback.

```
const handleSubmit = () => {
  if (username === "" || password === "") {
    Alert.alert('Missing Info', 'Fill in all fields.');
```

```
  } else {
    Alert.alert('Welcome', `Logged in as ${username}`);
  }
};
```

Building a Form – Full Example (p.3)

- The return statement renders a View holding both TextInputs.

```
return (
  <View>
    <TextInput value={username}
      onChangeText={setUsername} />
    <TextInput secureTextEntry value={password}
      onChangeText={setPassword} />
```

Building a Form – Full Example (p.4)

- Finally, add the Login button and close out the component.

```
    <Button title="Login" onPress={handleSubmit} />
  </View>
);
}
```

```
export default LoginForm;
```

Validating Form Input

- Validation checks that the data entered by the user is correct and complete before it is used or submitted.
- Simple validation can be done with if statements inside the submit handler:
 - Check for empty fields
 - Check minimum length (e.g., password of at least 6 characters)
 - Check that two values match (e.g., password and confirm password)

Giving Feedback with Alert

- Alert is a Core Component API that displays a native pop-up dialog with a title and message – useful for confirming actions or notifying the user after a form is submitted.
- Syntax:

```
Alert.alert(
  'Title', 'Message',
  [{ text: 'OK' }]
);
```

Key Takeaways

- TextInput captures user text input and can be customized with placeholder, secureTextEntry, and keyboardType.
- Events such as onPress and onChangeText let your app respond to what the user does.
- Button and TouchableOpacity handle taps; TouchableOpacity gives you full styling control.
- A form is a group of inputs managed with state and submitted with a Button.
- Alert delivers simple feedback and validation messages to the user.

ITWM106

Mobile Application Development 3 (cross-platform)

